程序设计思维与实践

课程笔记

大一秋季学期

2026 年 11 月 6 日

目录

1 第一节：常量与变量 **2**

1.1 常量 2

1.2 变量 2

2 第二节：简单的屏幕输出 **3**

3 第三节：数据类型 **3**

3.1 `#define` 与 `const` 的区别 5

4 第四节：如何计算变量或数据类型所占内存空间的大小 **5**

5 第五节：变量的赋值和赋值运算符 **6**

6 第一节：C 运算符和表达式 **6**

6.1 算术运算符和表达式 6

6.2 复合的赋值运算符 6

6.3 增 1 运算符和减 1 运算符 6

7 第二节： **7**

1 第一节：常量与变量

C 语言程序处理的数据有常量和变量两种形式。

1.1 常量

常量的实例

- 整型常量
- 实型常量
- 字符型常量
- 字符串字面量

不同进制的整型常量的表示形式

- 十进制
- 二进制
- 八进制
- 十六进制

- 不同类型的整型常量的表示形式
- 实型常量的表示形式
- 不同类型的实型常量的表示形式

1.2 变量

变量在使用之前必须先定义。定义变量的一般形式为：

类型关键字　　变量名；

变量名是用户定义的标识符，用于标识内存中一个具体的存储单元。
变量的命名应遵守以下基本的命名规则：

1. 标识符的第一个字符必须是字母（A-Z 或 a-z）或下划线 _。
2. 后续的字符可以是字母、数字（0-9）或下划线 _。

3. 标识符不能包含空格或其他特殊字符。

4. 不能使用 C 的关键词。

C 语言允许在定义变量的同时对变量初始化（为其赋初值）。

在一条语句中，可同时定义多个相同类型的变量，多个变量之间用逗号作分隔符。

若要在定义变量的同时将其初始化为 0，则为：

```
int a = 0 , b = 0 , c = 0 ;
```

但是不能写成：

```
int a = b = c = 0 ;
```

2 第二节：简单的屏幕输出

```
#include<stdio.h>
int main(void)
{
        printf(" ");
}
```

程序第一行以 # 开头而未以分号结尾的不是 C 语句，而是 C 的编译器处理命令。

尖括号内的文件称为头文件。编译器预处理命令可以使头文件在程序中生效。它的作用是：将写在尖括号内。

C 的标准输入/输出函数 `printf()` 的作用是输出一个字符串。

C 语言中，用一对双引号括起来的若干字符，称为字符串。

3 第三节：数据类型

C 语言中的数据类型分类。

这些类型的层次关系如下：

- 基本类型
  - 整型：基本整型 `int`、长整型 `long int`（`long`）、短整型 `short int`（`short`）、无符号整型 `unsigned`
  - 实型（浮点型）：单精度实型 `float`、双精度实型 `double`、长双精度实型 `long double`

表 1: C 语言数据类型分类（关键字、变量声明实例与讲述章节）

数据类型分类	关键字	变量声明实例	讲述章节
基本类型	基本整型 int	int a;	第 2 章
	长整型 long int	long int a;; 或 long a;	第 4 章
	长整型 long	long long int a;; 或 long long a;	第 4 章
	短整型 short int	short int a;; 或 short a;	第 4 章
	无符号整型 unsigned	unsigned int a; unsigned long b; unsigned short c;	第 4 章
	单精度实型 float	float a;	第 4 章
	双精度实型 double	double a;	第 4 章
	长双精度实型 long double	long double a;	第 4 章
	字符型 char	char a;	第 4 章、第 10 章
	枚举类型 enum	enum response{no, yes, none}; enum response answer;	第 12 章
构造类型	数组 --	int score[10]; char name[20];	第 8 章
	结构体 struct	struct date{ int year; int month; int day; }; struct date d;	第 12 章
	共用体 union	union{ int single; char spouseName[20]; struct date divorcedDay; }married;	第 12 章
指针类型	--	int *ptr; char *pStr;	第 9–11 章
空类型	void	void Sort(int array[], int n); void *malloc(unsigned int size);	第 8 章、第 11 章

  - 字符型 char
  - 枚举类型 enum
- 构造类型：数组类型、结构体类型 struct、共用体类型 union
- 指针类型（int *、char * 等）
- 空类型 void

3.1 #define 与 const 的区别

常量可以用 #define 预处理指令或 const 关键字定义，选择哪种方式取决于具体需求与编程习惯。通常情况下建议使用 const，因为它具有类型检查和作用域的优势；而 #define 仅做简单的文本替换，可能带来意外问题。

- **替换机制**：#define 进行简单的文本替换，const 则声明一个具有类型的常量。#define 定义的常量在编译时被直接替换为对应的值；const 定义的常量在程序运行时分配内存，并且具有类型信息。
- **类型检查**：#define 不做类型检查，因为它只是文本替换；const 定义的常量具有类型信息，编译器可对其做类型检查，有助于捕获潜在的类型错误。
- **作用域**：#define 定义的常量没有作用域限制，在定义之后的整个代码中都有效；const 定义的常量具有块级作用域，只在其定义所在的作用域内有效。
- **调试和符号表**：#define 定义的常量在符号表中没有对应条目，因为它只是文本替换；const 定义的常量在符号表中有对应条目，有助于调试和可读性。

```
#define MAX 100          /* 预处理阶段文本替换，无类型 */
const int max = 100;     /* 有类型、有块级作用域的常量 */
```

定义整型变量时，只要不指定为无符号型，其隐含的类型就是有符号型，而 signed 通常是省略不写的。

4 第四节：如何计算变量或数据类型所占内存空间的大小

衡量内存空间大小的表示单位。

sizeof 是 C 语言的关键字，不是函数名。sizeof() 是 C 语言提供的专门用于计算指定数据类型字节数的运算符。例如，计算 int 型数据所占内存的字节数用 sizeof(int) 计算即可。使用 sizeof(变量名) 的形式还可以计算一个变量所占内存的字节数。此外，sizeof 是编译时执行的运算符，不增加程序额外的运行时间开销，但是可以增强程序的可移植性。

5 第五节：变量的赋值和赋值运算符

赋值运算符用于给变量赋值。由赋值运算符及其两侧的操作数组成的表达式称为赋值表达式。

```
x = 1 ;
```

赋值操作是通过在赋值表达式后面加分号构成表达式语句来实现的。

赋值运算符的含义是将赋值运算符右侧表达式的值 (简称为右值) 赋给左侧的变量，即在 C 语言中赋值运算的操作是有方向性的，并无“等号两侧操作数的值相等”之意，因此，等号左侧只能是标识一个特定存储单元的变量名。

在计算含有不同类型运算符的表达式时，要考虑运算符的优先级，根据优先级确定运算的顺序，即先执行优先级高的运算，然后再执行优先级低的运算。

算术运算符的优先级高于赋值运算符的优先级。

在 C 语言中，赋值表达式的值被规定为运算完成后左操作数，类型与左操作数相同。

根据运算符的结合性来确定运算的顺序。运算的结合性有两种：一种是左结合，即自左向右计算；另一种是右结合，即自右向左计算。

C 语言中需要两个操作数的算术运算符是左结合的。

```
a = b = c = 0 ;
a = (b = ( c = 0)) ;
```

执行时是从右向左把最右侧的表达式的值依次赋值给左侧的变量。像上面形式的赋值表达式称为多重赋值表达式，一般用于为多个变量赋予相同的数值。

6 第一节：C 运算符和表达式

6.1 算术运算符和表达式

算术运算符的优先级与结合性。

余数的符号与被除数的符号相同。

6.2 复合的赋值运算符

复合的赋值运算符有 5 个，分别为

6.3 增 1 运算符和减 1 运算符

作为前缀运算符时，是在变量使用之前先对其执行加 1 操作。

7　第二节：